

PBA: 5. Causality II: DAGs and Covariate Selection

Fall 2026

Jaewon (“Jay-one”) Yoo

National Tsing Hua University

Where are we? Where are we going?

- **Where we have been**

- Weeks 1–3: R and RStudio, data visualization with ggplot2, data wrangling with dplyr
- Week 4: causality. Potential outcomes, randomized experiments, and three designs for observational data (Line Pay)

Where are we? Where are we going?

- **Where we have been**

- Weeks 1–3: R and RStudio, data visualization with `ggplot2`, data wrangling with `dplyr`
- Week 4: causality. Potential outcomes, randomized experiments, and three designs for observational data (Line Pay)

- **Today: causality II**

- Causal diagrams (DAGs): which variables to control for, and which not to
- Building on last week: the confounding picture and `group_by()` comparisons, with data we simulate ourselves, so we know the true effect and can see which comparison gets it right
- After the lecture, run today's code yourself and start sketching your team's DAG

Where are we? Where are we going?

- **Where we have been**

- Weeks 1–3: R and RStudio, data visualization with `ggplot2`, data wrangling with `dplyr`
- Week 4: causality. Potential outcomes, randomized experiments, and three designs for observational data (Line Pay)

- **Today: causality II**

- Causal diagrams (DAGs): which variables to control for, and which not to
- Building on last week: the confounding picture and `group_by()` comparisons, with data we simulate ourselves, so we know the true effect and can see which comparison gets it right
- After the lecture, run today's code yourself and start sketching your team's DAG

- **Where we are going**

- Week 6 (Oct 15): no class meeting. Team meetings by appointment: bring your research question, its context, and your DAG
- Week 7 (Oct 22): relationships between variables, importing and tidying data. Then prediction, regression, and uncertainty

Roadmap

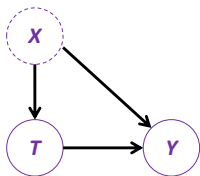
1. DAGs: pictures of what causes what
2. Reading a DAG: forks, chains, colliders
3. Choosing controls for your project



Sources: <https://www.redbubble.com/i/sticker/D-Ya-Like-Dags-by-salamincheese/27407958.EJUG5>

p.s.: from an old British-American comedy film starring Brad Pitt

Last Week: Confounding in the Line Pay Data



- X: tech-savviness, T: Line Pay adoption, Y: m-commerce spending
 - Adopters vs. non-adopters, after the launch: **+23.4**
 - Within tech-savviness groups (subclassification): **+20.1** and **+19.0**
- This picture is already a **causal diagram**: arrows say what causes what.
 - Comparing within groups of X removed the part of the gap that came through X.
 - Today: the grammar of such pictures, so that you can decide **which variables to compare within, and which ones to leave alone.**

Today's Questions

- Which variables should we **control for**?
- Which variables should we **not** control for, even though they are in the data?
- What is a **pre-treatment** variable, and is “pre-treatment” enough?
- A dataset with dozens of columns: **how do we choose**?

1/ DAGs: pictures of what causes what

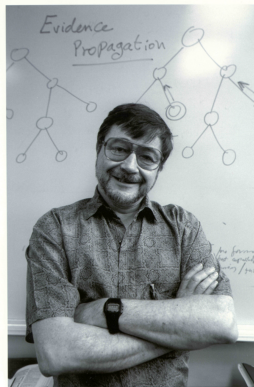
Recap: Potential Outcomes

Last week we defined causal effects as **contrasts between potential outcomes**:

$$Y_i(1) - Y_i(0), \quad \text{SATE} = \frac{1}{n} \sum_{i=1}^n (Y_i(1) - Y_i(0))$$

- We never see both $Y_i(1)$ and $Y_i(0)$ for the same unit (fundamental problem).
- Randomization makes treated and control groups comparable by *design*.
- Without randomization, we need assumptions: e.g., *comparable within groups of tech-savviness*.
- Today: an **alternative causal language**, directed acyclic graphs (DAGs).
 - Same goal (learn $Y_i(1) - Y_i(0)$ on average), different way of writing the assumptions.

Judea Pearl and Causal Graphs



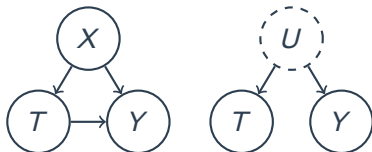
Judea Pearl (1936–) turned causal diagrams into a formal language of causality.

- Originally a computer scientist (artificial intelligence)
- *Causality* (2000, Cambridge UP)
- Turing Award in 2011 for this work

Key idea: **no functional form**. A graph only says *which* variables each variable depends on, not *how*.

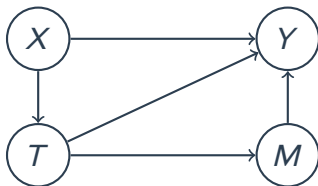
Directed Acyclic Graphs

- **Directed acyclic graphs** (DAGs) describe the causal structure of variables



- **Nodes:** variables, observed (solid) or unobserved (dashed).
- **Arrows:** a possible direct causal effect.
 - Arrow $T \rightarrow Y$: $Y_i(1) \neq Y_i(0)$ for at least some units.
 - **No arrow is an assumption:** no direct effect at all. Missing arrows are the strong claims.
- **Directed:** each arrow has a direction (cause $\rightarrow$ effect).
- **Acyclic:** no cycles. A variable cannot cause itself.

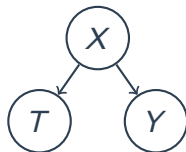
DAG Terminology



- **Path:** a sequence of arrows connecting two nodes, in any direction.
 - **Causal (directed) path:** all arrows point the same way, e.g., $T \rightarrow M \rightarrow Y$.
 - **Non-causal path,** e.g., $T \leftarrow X \rightarrow Y$.
- **Parents** and **children:** T is a parent of M and Y ; M is a child of T .
- **Ancestors** and **descendants:** M and Y are descendants of T and X ; X is an ancestor of M .

A DAG Is a Recipe for Data

```
1 > set.seed(1008)
2 > n <- 2000
3 > tech <- rbinom(n, 1, 0.5)           # X: no parents
4 > adopt <- rbinom(n, 1, 0.3 + 0.4 * tech) # T: parent X
5 > spend <- 50 + 20 * tech + rnorm(n, 0, 10) # Y: parent X only
6 > fork <- tibble(tech, adopt, spend)
```



- `rbinom`(n , 1, p): n zeros and ones, each 1 with probability p (a weighted coin). `rnorm`(n , 0, 10): random noise around 0.
- **One line per node**: each variable is made from its **parents** plus its own noise.
- **Arrows** = variables used on the right-hand side. `adopt` is never used to make `spend`, so there is **no arrow** $T \rightarrow Y$.
- In this simulated world, Line Pay has **zero effect** on spending, and we know it.
- Real data come from a recipe we never see. Here we write the recipe, so we know the true effect and can see which comparison (overall or within groups) gets it right.

Q: Draw the DAG

Recipe A

```
adopt <- rbinom(n, 1, 0.5)
coupon <- rbinom(n, 1, 0.2 + 0.5 * adopt)
spend <- 50 + 15 * coupon + rnorm(n, 0, 10)
```

Recipe B

```
adopt <- rbinom(n, 1, 0.5)
spend <- 50 + rnorm(n, 0, 10)
member <- if_else(adopt == 1 | spend > 60,
                  1, 0)
```

Q: Draw the DAG

Recipe A

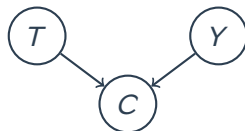
```
adopt <- rbinom(n, 1, 0.5)
coupon <- rbinom(n, 1, 0.2 + 0.5 * adopt)
spend <- 50 + 15 * coupon + rnorm(n, 0, 10)
```



M: uses in-app coupons

Recipe B

```
adopt <- rbinom(n, 1, 0.5)
spend <- 50 + rnorm(n, 0, 10)
member <- if_else(adopt == 1 | spend > 60,
                  1, 0)
```

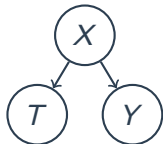


C: joins the loyalty program (if adopted
Line Pay or spends over 60)

2/ Reading a DAG: forks, chains, colliders

Three Building Blocks

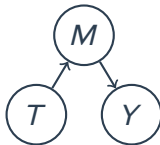
Fork



X = **confounder**

common cause

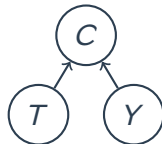
Chain



M = **mediator**

on the causal path

Inverted fork



C = **collider**

common effect

- Every larger DAG is built from these three pieces.
- For each one, two questions:
 1. Do T and Y move together in the data?
 2. Do they still move together once we **compare within groups** of the middle variable?

What Does “Independent” Mean?

- T and Y are **independent**, written $T \perp\!\!\!\perp Y$, if knowing T tells us nothing about Y .
 - Quick check with last week's tools: **does the average of Y differ between $T = 1$ and $T = 0$?**
- **Conditionally independent** given X , written $T \perp\!\!\!\perp Y \mid X$: independent *within each group* of X .
 - “Conditioning on X ” = “controlling for X ” = comparing within groups of X (Week 4: subclassification).
- Strictly, independence concerns the whole distribution, not just the average; the average is our quick check for today.

Fork in Data: Overall Comparison

- Recipe from before: tech-savvy customers adopt more *and* spend more; Line Pay itself does nothing.

```
1 > fork |>
2   group_by(adopt) |>
3   summarize(avg_spend = mean(spend))
4
5 # A tibble: 2 × 2
6   adopt avg_spend
7   <int>     <dbl>
8 1     0      55.8
9 2     1      64.2
```

- Adopters spend **8.4 more**, although adoption has **no effect** in this recipe.
- Q: Where does the gap come from?

Fork in Data: Compare Within Tech Groups

```
1 > fork |>
2   group_by(tech, adopt) |>
3   summarize(avg_spend = mean(spend)) |>
4   pivot_wider(names_from = adopt, values_from = avg_spend,
5               names_prefix = "adopt_") |>
6   mutate(diff = adopt_1 - adopt_0)
7
8 # A tibble: 2 × 4
9 # Groups:   tech [2]
10   tech adopt_0 adopt_1 diff
11   <int>   <dbl>   <dbl> <dbl>
12 1      0    50.3    50.1 -0.230
13 2      1    69.8    69.9  0.0786
```

- Within each tech group the gap is **close to zero**: $T \perp\!\!\!\perp Y \mid X$.
- **Fork: controlling for the confounder removes a gap that is not causal.** This is what subclassification did for Line Pay last week.

Chain in Data: Overall Comparison

```
1 > set.seed(1008)
2 > adopt <- rbinom(n, 1, 0.5)           # T: no parents
3 > coupon <- rbinom(n, 1, 0.2 + 0.5 * adopt) # M: parent T
4 > spend <- 50 + 15 * coupon + rnorm(n, 0, 10) # Y: parent M only
5 > chain <- tibble(adopt, coupon, spend)
6
7 > chain |>
8   group_by(adopt) |>
9   summarize(avg_spend = mean(spend))
10
11 # A tibble: 2 × 2
12   adopt avg_spend
13   <int>   <dbl>
14 1     0    53.0
15 2     1    60.4
```

- Here Line Pay **does** raise spending, through coupon use: adopters use more coupons, and coupon users spend more.
- The gap of 7.4 is a real causal effect (the recipe implies $0.5 \times 15 = 7.5$).

Chain in Data: Compare Within Coupon Groups

```
1 > chain |>
2   group_by(coupon, adopt) |>
3   summarize(avg_spend = mean(spend)) |>
4   pivot_wider(names_from = adopt, values_from = avg_spend,
5               names_prefix = "adopt_") |>
6   mutate(diff = adopt_1 - adopt_0)
7
8 # A tibble: 2 × 4
9 # Groups:   coupon [2]
10  coupon adopt_0 adopt_1 diff
11   <int>   <dbl>   <dbl> <dbl>
12 1      0    50.3    49.8 -0.522
13 2      1    65.0    64.9 -0.135
```

- Within coupon groups the gap **disappears**, yet the effect is real.
- Chain: controlling for the mediator removes the causal effect you were looking for. A “bad control” when the target is the total effect.

Collider in Data: Overall Comparison

```
1 > set.seed(1008)
2 > adopt <- rbinom(n, 1, 0.5)           # T: no parents
3 > spend <- 50 + rnorm(n, 0, 10)        # Y: no parents
4 > member <- if_else(adopt == 1 | spend > 60, 1, 0) # C: parents T and Y
5 > collider <- tibble(adopt, spend, member)
6
7 > collider |>
8   group_by(adopt) |>
9   summarize(avg_spend = mean(spend))
10
11 # A tibble: 2 × 2
12   adopt avg_spend
13   <int>   <dbl>
14 1     0    49.7
15 2     1    50.1
```

- No effect in the recipe, and **no gap** in the data: $T \perp\!\!\!\perp Y$.
- Now suppose the only data we can get come from a partner's **loyalty program**.

Collider in Data: Analyze Members Only

```
1 > collider |>
2   filter(member == 1) |>
3   group_by(adopt) |>
4   summarize(avg_spend = mean(spend), n = n())
5
6 # A tibble: 2 × 3
7   adopt avg_spend      n
8   <int>    <dbl> <int>
9 1     0     65.2   144
10 2     1     50.1   985
```

- Among members, adopters spend **15.1 less**. A gap **from nothing**.
- Why: a non-adopter is a member *only if* they spend over 60. Selecting on the collider links its two causes.
- `filter()` on a collider is conditioning too. Membership is affected by adoption, so this is **post-treatment bias**.

Collider in Data: The Same Customers, Two Views



- Black diamonds: group averages. Keeping only members removes the low-spending non-adopters.

Three Structures, Three Patterns

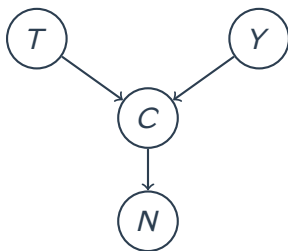
Structure	Overall gap	Within groups	Advice
Fork $T \leftarrow X \rightarrow Y$	yes (not causal)	gone	control for X
Chain $T \rightarrow M \rightarrow Y$	yes (causal)	gone	do not control for M
Collider $T \rightarrow C \leftarrow Y$	none	appears	do not control or select on C

- A path through a fork or a chain is **open** until we condition on the middle node.
- A path through a collider is **blocked** until we condition on the collider.
- These two rules, applied to every path, are called **d-separation**.

d-Separation

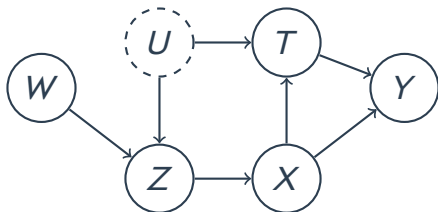
- Can we read (conditional) independence off a DAG? **Yes**. To check $A \perp\!\!\!\perp B \mid C$:
 1. Find all paths between A and B .
 2. Check whether each path is **blocked**, given that we condition on C .
 3. If every path is blocked, A and B are **d-separated** by C , and $A \perp\!\!\!\perp B \mid C$.
- A path is **blocked** given C if
 1. it contains a **non-collider** (fork or chain middle) that is in C , **or**
 2. it contains a **collider** that is not in C and has no descendant in C .
- If some path stays open, A and B are **d-connected**: expect them to move together.

Descendants of a Collider Count Too



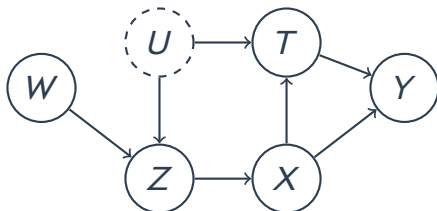
- C : loyalty member. N : receives the members' newsletter (only members get it).
- Analyzing only newsletter recipients = selecting on a descendant of the collider.
- The path $T \rightarrow C \leftarrow Y$ opens just as if we had selected on C .
- In practice: ask **how a row got into your dataset**. Survey responders, app reviewers, surviving firms, customers who log in.

d-Separation Example



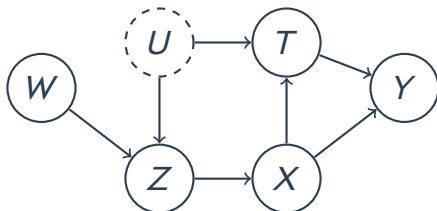
- Q1: Are W and Y independent of each other?

d-Separation Example



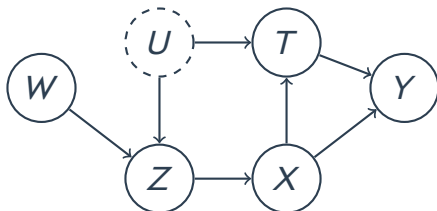
- Q1: Are W and Y independent of each other?
- No.
 - Blocked at the collider Z : $W \rightarrow Z \leftarrow U \rightarrow T \rightarrow Y$,
 $W \rightarrow Z \leftarrow U \rightarrow T \leftarrow X \rightarrow Y$
 - Open: $W \rightarrow Z \rightarrow X \rightarrow Y$, $W \rightarrow Z \rightarrow X \rightarrow T \rightarrow Y$

d-Separation Example



- Q1: Are W and Y independent of each other?
- No.
 - Blocked at the collider Z : $W \rightarrow Z \leftarrow U \rightarrow T \rightarrow Y$,
 $W \rightarrow Z \leftarrow U \rightarrow T \leftarrow X \rightarrow Y$
 - Open: $W \rightarrow Z \rightarrow X \rightarrow Y$, $W \rightarrow Z \rightarrow X \rightarrow T \rightarrow Y$
- Q2: Which variables should we condition on to make W and Y d-separated?

d-Separation Example

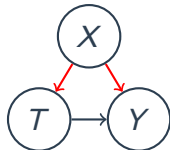


- Q1: Are W and Y independent of each other?
- No.
 - Blocked at the collider Z : $W \rightarrow Z \leftarrow U \rightarrow T \rightarrow Y$,
 $W \rightarrow Z \leftarrow U \rightarrow T \leftarrow X \rightarrow Y$
 - Open: $W \rightarrow Z \rightarrow X \rightarrow Y$, $W \rightarrow Z \rightarrow X \rightarrow T \rightarrow Y$
- Q2: Which variables should we condition on to make W and Y d-separated?
- $\{X, T\}$. X blocks both open paths but, as a descendant of the collider Z , **opens** $W \rightarrow Z \leftarrow U \rightarrow T \rightarrow Y$; T blocks that path.

3/ Choosing controls for your project

Backdoor Paths

- The causal effect of T on Y travels along the **causal paths** out of T .
- A **backdoor path** is a non-causal path from T to Y that starts with an arrow *into* T .
 - It would remain even if we deleted every arrow coming out of T .
 - Open backdoor paths come from **common causes** of T and Y .



- Line Pay: $T \leftarrow X \rightarrow Y$ (adoption $\leftarrow$ tech-savviness $\rightarrow$ spending).
- The overall gap mixes the causal path $T \rightarrow Y$ with the open backdoor path.

The Backdoor Criterion

A set of variables S is enough to control for if

1. no variable in S is a **descendant of T** (no post-treatment variables),
and
2. S **blocks every backdoor path** from T to Y .

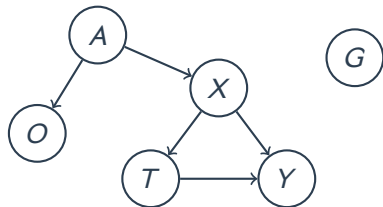
- Then treated and control units are comparable *within groups of S* :

$$(Y_i(1), Y_i(0)) \perp\!\!\!\perp T_i \mid S_i$$

This is last week's "as-if randomized within tech-savviness groups," now with a reason attached.

- The criterion answers three questions at once:
 1. Is there confounding, given this DAG?
 2. Can it be removed with the variables we have?
 3. Which variables should we control for?

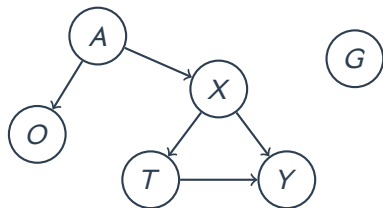
Line Pay, Revisited



- Q: Which variables satisfy the backdoor criterion?

Our assumed DAG for the linepay data.
X: tech-savviness, T: adoption, Y: spending after the launch, A: age, O: occupation, G: gender.

Line Pay, Revisited



Our assumed DAG for the lineup data.
X: tech-savviness, T: adoption, Y: spending after the launch, A: age, O: occupation, G: gender.

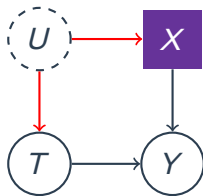
- Q: Which variables satisfy the backdoor criterion?
- tech_savviness: **yes**, it blocks $T \leftarrow X \rightarrow Y$.
- gender: **no**, not on any backdoor path.
- age: **no** on its own. It causes X but does not sit on $T \leftarrow X \rightarrow Y$. Once X is in, age adds nothing.
- occupation_type: **no**, it leads nowhere near Y .

Line Pay, Revisited: Check With the Data

```
1 > LinePay_data |>
2   group_by(gender, line_pay_adopt) |>
3   summarize(avg_spending = mean(spending_post)) |>
4   mutate(line_pay_adopt = if_else(line_pay_adopt == 1, "adopted", "unadopted")) |>
5   pivot_wider(names_from = line_pay_adopt, values_from = avg_spending) |>
6   mutate(diff = adopted - unadopted)
7
8 # A tibble: 2 × 4
9 # Groups:   gender [2]
10   gender unadopted adopted  diff
11   <int>    <dbl>    <dbl> <dbl>
12 1      0      59.6      82.3  22.7
13 2      1      58.9      82.9  24.0
```

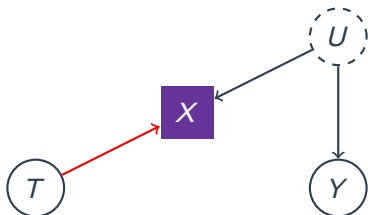
- Within gender groups: **22.7** and **24.0**, still close to the overall 23.4.
- Within tech-savviness groups (last week): **20.1** and **19.0**.
- The linepay data were simulated with an effect of **+20** for every adopter. Only the control the DAG pointed to gets there.

Can't Measure the Confounder? Block the Path



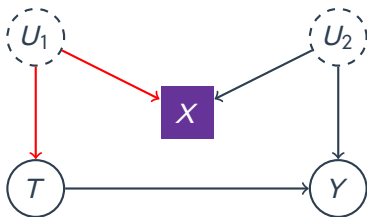
- T : job training program. Y : landing a job. U : motivation (unobserved). X : number of applications sent.
- Backdoor path $T \leftarrow U \rightarrow X \rightarrow Y$. We cannot measure U , but conditioning on X (shaded) **blocks** the path.
- **Big assumption**: no arrow $U \rightarrow Y$. Otherwise no observed variable blocks $T \leftarrow U \rightarrow Y$.

Bad Control: Post-Treatment Variables



- T : adopts Line Pay. X : app log-ins in the month *after* adoption. U : shopping enthusiasm (unobserved). No arrow $T \rightarrow Y$.
- X is a collider on $T \rightarrow X \leftarrow U \rightarrow Y$. Controlling for X **opens** the path: a gap appears where there is no effect.
- A **pre-treatment variable** is determined *before* the treatment, so the treatment cannot affect it.
- Rule of thumb: **never control for something the treatment can change**. That covers mediators (the chain) and post-treatment colliders (this slide).

Is Pre-Treatment Always Safe? M-Bias



- Without conditioning, $T \leftarrow U_1 \rightarrow X \leftarrow U_2 \rightarrow Y$ is **blocked** by the collider X .
- **Conditioning on X opens it**, even though X is measured before treatment. (The graph's shape gives the name.)
- Ex: T = schooling, Y = earnings, U_1 = family background, U_2 = ability, X = a test score.
- Debated: Rubin sees it as a rare curiosity; Pearl as a real threat. The point for us: **structure decides, not timing alone**.

Good Controls, Bad Controls

“Should I control for this variable?” has no one-size-fits-all answer.

Cinelli, Forney, & Pearl (2022) sort the answers by the variable's **role in the DAG**:

Type	Role in the DAG	Effect of controlling for it
Good	Confounder (common cause of T , Y)	Removes confounding bias
Neutral	Cause of Y only	No bias; more precise estimates
Neutral	Cause of T only	No bias; less precise estimates
Bad	Collider (common <i>effect</i> of T , Y)	Opens a non-causal path
Bad	Mediator on $T \rightarrow Y$	Removes part of the effect
Bad	Descendant of Y	Post-treatment bias

- Good or bad is a **structural** question: it depends on the DAG, not on how the data look.

A Dataset With Many Columns: Which Ones?

- For each candidate variable, ask:

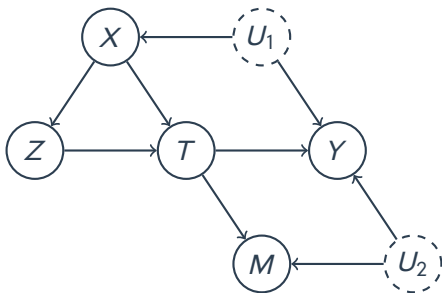
	Before T ?	Causes T ?	Causes Y ?	Decision
Confounder	yes	yes	yes	control
Cause of Y only	yes	no	yes	optional (precision)
Cause of T only	yes	yes	no	leave out
Changed by T or Y	no	(either)		do not control

- Then draw the DAG with the survivors and check the backdoor criterion: M-shaped structures can still bite.
- “Include everything we have” is not a strategy. Neither is “include what makes the estimate look good.”

Where Does the DAG Come From?

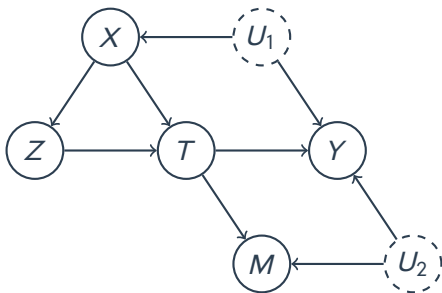
- Not from the data. From **theory, domain knowledge, and how the data were produced.**
- A practical workflow:
 1. **Nodes:** T , Y , and every variable that plausibly causes either (unobserved $\rightsquigarrow$ dashed).
 2. **Arrows:** for each pair, “could A directly cause B ?” Missing arrows are the strong assumptions.
 3. **Direction:** time order and common sense.
 4. **How rows enter the data:** any selection step is a node too.
- Drawing the DAG “forces a clearer articulation of the research question” (Tafti & Shmueli 2025, *MIS Quarterly*).
- Not sure about an arrow? Draw both versions and see whether your answer changes.

Practice: The Backdoor Criterion by Hand



1. List all **backdoor paths** between T and Y .
2. List the sets of *observed* nodes that **block** all of them.
3. Drop the sets that include a **descendant of T** .

Practice: The Backdoor Criterion by Hand



1. List all **backdoor paths** between T and Y .
 2. List the sets of *observed* nodes that **block** all of them.
 3. Drop the sets that include a **descendant of T** .
- Backdoor paths: $T \leftarrow X \leftarrow U_1 \rightarrow Y$ and $T \leftarrow Z \leftarrow X \leftarrow U_1 \rightarrow Y$.
Blocking sets: $\{X\}$, $\{X, Z\}$. Leave M out: it is a descendant of T , and a collider.

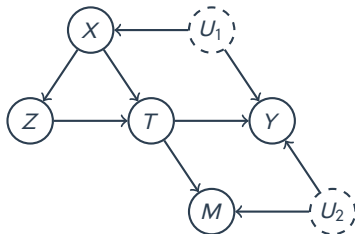
Why Let the Computer Do It?

Backdoor analysis by hand is fine for five nodes. Project DAGs are bigger.

- **Scale:** with 15 nodes, the number of paths grows quickly.
- **Correctness:** people miss paths where colliders and chains interact.
- **Reproducibility:** a DAG written as code can be checked by your teammates and by me.
- **Robustness:** change one arrow, rerun, see whether the answer survives.

R Package: dagitty

```
1 # install.packages("dagitty") # once
2 > library(dagitty)
3 > g <- dagitty('dag {
4     X -> Z -> T -> Y
5     X -> T -> M
6     M <- U2 -> Y
7     X <- U1 -> Y
8 }')
9 > latents(g) <- c("U1", "U2") # unobserved
```



- Each line lists arrows; $X \rightarrow Z \rightarrow T \rightarrow Y$ is three arrows at once.
- `plot(g)` draws it; <https://www.dagitty.net> draws it in the browser.

R Package: dagitty

- Parents, ancestors, children, descendants:

```
1 > parents(g, "T")
2 [1] "X" "Z"
3 > descendants(g, "T")
4 [1] "T" "Y" "M"
```

- All paths between T and Y , and only the causal ones:

```
1 > paths(g, "T", "Y")$paths
2 [1] "T -> M <- U2 -> Y"      "T -> Y"      "T <- X <- U1 -> Y"
3 [4] "T <- Z <- X <- U1 -> Y"
4 > paths(g, "T", "Y", directed = TRUE)$paths
5 [1] "T -> Y"
```

- d-separation:

```
1 > dseparated(g, "Z", "M", c("T"))
2 [1] TRUE
```

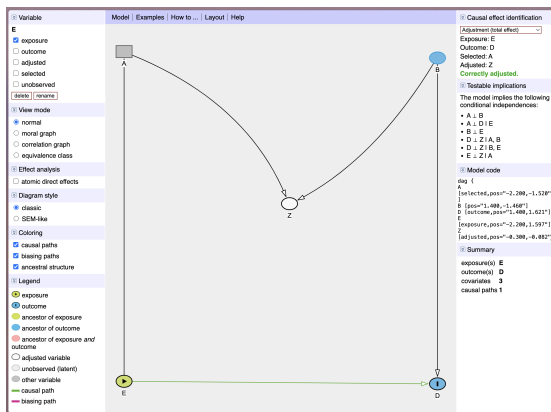
R Package: dagitty

- What to control for (sets that satisfy the backdoor criterion):

```
1 > adjustmentSets(g, "T", "Y")  
2 { X }
```

- The same answer we found by hand. `type = "all"` lists every valid set; $\{X, Z\}$ is one of them.
- Which valid set to use?
 - Sets with unobserved variables are not available.
 - The smallest observed set is the cheapest to collect and is enough for identification.
 - Adding a cause of Y can make estimates more precise (regression weeks).

DAGitty in the Browser (dagitty.net)



- Draw interactively; valid adjustment sets are highlighted as you go; exports the code for R.

In-Class Practice: Your Team's DAG

1. Write your question as “Does T change Y ?” Name T and Y as variables you could measure.
 - No question yet? Start from the guide to finding data:
<https://j1yoo.github.io/teaching/pba-data/>
2. List the variables that plausibly cause T or Y . Mark the ones you cannot observe (dashed).
3. Draw it on <https://www.dagitty.net>. Draw an arrow only where you believe a direct effect exists.
4. Ask how rows enter your data. If there is a selection step, add it as a node.
5. Sort each variable as in the good-control and bad-control tables; then read off the adjustment set.
6. Write one sentence on the unobserved variable that worries you most.
 - **Week 6 meeting: bring your research question, its context, and a picture of your DAG that you can walk me through.**

What a DAG Cannot Do

- A DAG is only as good as its assumptions. Data can reject some of them, but never confirm the whole graph.
- If an **unobserved** confounder sits on a backdoor path, no set of observed controls is enough.
 - Then we need designs that handle confounding differently: experiments, or designs such as last week's difference-in-differences.

Summing Up

- A DAG is an **alternative causal language** to potential outcomes: same goal, different way of stating assumptions.
- Three building blocks: **fork** (control for it), **chain** (do not, if you want the total effect), **collider** (do not control or select on it).
- **d-separation** applies these rules to every path; the **backdoor criterion** turns them into a list of controls.
- “Should I control for X?” is a **structural** question, answered by the DAG, not by the data.
- `group_by()` and `filter()` are conditioning. So is choosing which rows enter your data.

What Comes Next?

- Today: causal diagrams, three building blocks, d-separation, and the backdoor criterion.
- Week 6 (Oct 15): **no class meeting**. Team meetings by appointment: bring your research question, its context, and your DAG.
 - No question yet? Start from the guide to finding data:
<https://j1yoo.github.io/teaching/pba-data/>
- Week 7 (Oct 22): **relationships between variables**. Correlation measures how variables move together; today's DAG tells you what that movement can and cannot mean.
- **Assignment 2 is released today**. Milestone 1 (GitHub repository) is due Sunday, Oct 11; Milestone 2 (data and proposal) is due Sunday, Oct 25.
- Before your Week 6 meeting: Huntington-Klein, *The Effect*, Ch. 7 (drawing causal diagrams; free online).

Enjoy your Weekends! :)

Contact Information:

jaewon.yoo@iss.nthu.edu.tw

<https://j1yoo.github.io/>

